WNDRVR
SUPPORT NETWORK

VXWORKS 7 USB CORE PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 USB Core Programmer's Guide, SR0650

13 August 2020

1. USB CORE OVERVIEW

1. USB Technology

The VxWorks® 7 USB Core Programmer's Guide describes the use of the VxWorks USB. The VxWorks USB supports the Universal Serial Bus Specification (USB), Revision 3.0, which provides hosts and devices with a versatile channel for communication.

USB is available at low (1.5 Mbps), full (12 Mbps), high (480 Mbps) and super speed (5 Gbps) data transfer rates, and allows for the following types of transfers:

- control
- bulk (including Stream)
- interrupt
- isochronous

USB also incorporates provisions for power management and for the dynamic attachment and removal of devices.

2. Wind River USB

Wind River USB allows you to build a VxWorks bootable image with USB support for a host stack, a target stack, and an OTG stack.

Each of the three USB stacks provides unique support for USB:

- The host stack enables VxWorks to use USB devices.
- The target stack allows a host machine, such as a PC, to treat a VxWorks machine as a USB device.
- The OTG stack supports USB On-The-Go (USB OTG) with dynamic role switching capability.

USB flexibility allows it to be used, often concurrently, by different kinds of devices, each kind requiring its own device driver support. It is desirable for these device drivers to be written so that they are independent of each other and independent of the implementation of the host computer's underlying USB host controller interface. Wind River USB meets these requirements by providing a complete set of services to operate the USB, and a number of prebuilt USB class drivers, each of which handles one kind of USB device.

Architecture and BSP Support

The USB host and target stacks support several target architectures. Wind River also provides support for the USB host and target stacks as standard parts for many board support packages (BSPs).

For more information on available architectures and BSPs, see your product release notes.

SMP Support

Wind River USB provides SMP facilities that support symmetric multiprocessing (SMP), and that make it SMP-ready, meaning that it runs correctly on SMP hardware, although it may not make use of more than one CPU.

Documentation

The Wind River USB documentation assumes that you are already familiar with the USB specification, Workbench, and the VxWorks operating system.

Wind River USB has been developed in compliance with the *Universal Serial Bus Specification*, Revision 3.0, generally referred to in this document as the "USB specification." Where possible, the documentation uses terminology similar to that used in the USB specification so that the correspondence between USB concepts and actions is apparent in the software interfaces described.

3. Additional Documentation

Additional documentation about USB is available from several sources.

Reference Information

Workbench help content contains a **VxWorks-7 API and Configuration Reference > Feature: CONNECTIVITY > USB** entry.

Migration Information

For information on migration, see Migration and Porting Topics on page 9.

USB-Specification Information

For general specification information, refer to the following source.

- *Universal Serial Bus Specification*, Rev. 2.0, April 27, 2000. All USB specifications are available at: http://www.usb.org/developers/docs

For host controller specifications, refer to the following sources.

- for EHCI:
  http://www.intel.com/technology/usb/download/ehci-r10.pdf
- for OHCI:
  https://stuff.mit.edu/afs/sipb/contrib/doc/specs/protocol/usb/hcir1_0a.pdf
- for UHCI:
  ftp://ftp.netbsd.org/pub/NetBSD/misc/blymn/uhci11d.pdf
- for xHCI:
  http://www.intel.com/content/www/us/en/io/universal-serial-bus/extensible-host-controler-interface-usb-xhci.html?wapkw=xhci
- for MhdrcHCI:
  http://www.mentor.com/products/ip/usb/usb20otg/upload/MUSBMHDRC__Datasheet.pdf
- for SynopsysHCI:
  https://www.synopsys.com/dw/ipdir.php?c=dwc_usb_2_0_host_subsystem-pci-ahb

> Note: To access the SynopsysHCI data sheet you must register for a user login on the Synopsis website. Contact your Synopsis representative for more details.

For USB device class and vendor specifications, refer to the following resources.

- *Communications Device Class - CDC Subclass Specification for PSTN Devices, Abstract Control Model.*
- *USB Device Class Definition for Human Interface Devices (HID)* - provides support for USB keyboard and mouse)), Rev. 1.11, June 27, 2001

For information on the mass storage class, refer to the following resources.

- *USB Mass Storage Class Specification Overview, Rev. 1.2*, June 23, 2003
- *USB Mass Storage Class Bulk Only Transport*, Rev. 1.0, September 31, 1999
- *USB Mass Storage Class Control/Bulk/Interrupt (CBI) Transport*, Rev. 1.1, June 23, 2003
- *USB Mass Storage Class UFI Command Specification*, Rev. 1.0, December 14, 1998

For information on the printer class, refer to the following source.

- *USB Device Class Definition for Printing Devices*, Rev. 1.1, January 2000

For information on the USB networking devices, refer to the following sources.

- Asix USB networking chipset series (Asix88772, Asix88172, etc.)
  http://www.asix.com.tw/index.php?&width=1680
- Dm960x USB networking chipset series (Dm9601, etc.)
  http://www.davicom.com.tw
- Pegasus USB networking chipset series (ADM8511, ADM8515/L, ADM8513)
  http://www.infineon.com

For information on the On-The-Go specification, refer to the following source.

- USB On-The-Go specification: https://www.usb.org/sites/default/files/documents/usb_otg_and_eh_3-0_release_1_1_10may2012.pdf

Target Stack Information

For additional information relevant to the USB peripheral stack, refer to the following sources:

- *Universal Serial Bus Specification* Rev. 2.0, and USB device class specifications.
- *Firmware Programming Guide for NET2280 PCI USB High Speed Peripheral Controller*, Rev. 1A.

4. VxWorks Source Build Options for USB Support

Your Wind River VxWorks 7 installation includes source code, but does not include pre-built kernel libraries.

Before beginning any VxWorks development, you must build the VxWorks kernel libraries. You must also build the VxWorks run-time kernel image based on the kernel libraries.

To configure and build USB, include the appropriate components, as documented for each package. For details on how to configure a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For more information on USB components and parameters, see the *Wind River USB Component Reference*.

VSB Options

The USB VSB options are listed below. The VSB option is required to build a VSB with the corresponding support.

VSB Option	Description
USB	Container only.
USB_CORE	Wind River USB core package.

VSB Option	Description
USB_HOST	The USB host stack package.
USB_CTLR	Container only.
USB_CCORE	The host controller core support.
USB_MHDRC	The Mentor Graphics package.
USB_DWC2DR	The Synopsys DesignWare series USB 2.0 host controller package.
USB_FSLDR	The device mode of Freescale USB Dual-Role controller package.
USB_PCHUDC	The Intel USB 2.0 device controller package.
USB_EHCI	The EHCI host controller package.
USB_OHC	The OHCI host controller package.
USB_UHCI	The UHCI host controller package.
USB_XHCI	The xHCI host controller package.
USB_CLASS	The class driver core package.
USB_HELPER	This package includes **usb2Helper.c**, **usb2Helper.h**, and **usrUsb2HelperInit.c**. It creates a common library for all class drivers such as add/remove, create/delete mechanism. These are common libraries and services for the class driver core package.
USB_HID	The HID core package.
USB_KEYBOARD	The keyboard class driver package.
USB_TOUCHSCREEN	The touchscreen class driver package.
USB_MOUSE	The mouse class driver package.
USB_PRINTER	The printer class driver package.
USB_STORAGE	The mass storage class driver package.
USB_NETWORK	The network driver package.
USB_SERIAL	The serial class driver package.

VSB Option	Description
USB_HCORE	The USB host core package, which defines key structures, macro, and functions for creating host libraries. It is used by both the host controller drivers and class drivers.
USB_OTG	The OTG stack package.
USB_PHY	Used with the the host controllers.
USB_TARGET	Container only.
USB_TCORE	The target stack core package.
USB_FUNCTION	The target function driver core package.
USB_HLP	Common library and services for function drivers package.
USB_KBD	The keyboard target function driver package.
USB_MSC	The mass storage function driver package.
USB_NET	The network target function driver package.
USB_PRNT	The printer function driver package.
USB_SER	The serial function driver package.
USB_PLX	The PLX target controller driver package.
USB_UVC	USB video class driver.

USB Bootroms

For information on configuring a USB bootrom, refer to the VxWorks 7 itl_generic BSP Supplement Guide.

2. DEBUGGING USB

1. Using the USB SHOW Routines

USB SHOW provides a command-line tool, similar to **usbutils**, which allows users and applications to discover what USB peripherals are connected to the VxWorks host.

Procedure

1. Define **INCLUDE_USB_SHOW** in your project.
2. Set the **USB_SHOW_LEVEL** parameter value. There are three levels of support:

   0 - This is a very simple listing of the USB ports

   1 - Listing of all the USB ports with descriptor dumps. This can be a lot of output.

   2 - This is the default setting. It includes Level 1 information along with the addition of port status information.

3. Build your image and start it.
4. Type **usbShow** in the shell.

2. Using the USB TRACK Tool

USB TRACK supports the host stack and outputs trace messages to enhance USB debugging of your class drivers. Here take hub class driver debug as an example.

Procedure

1. Enable the USB debug VSB option in your VSB project.

   wrtool prj vsb config -w -add _WRS_CONFIG_USB_DEBUG_ENABLE=y

2. Add the USB host core layer **USB_HCORE** to your VSB project.

   wrtool prj vsb add USB_HCORE

3. Build your VxWorks source build (VSB) project.
4. Create a VxWorks image project (VIP) project based on your VSB project.
5. Include **INCLUDE_USB_TRACK_HUB** in your VIP.
6. Select the desired USB HUB debug channel in for the **USB_TRACK_HUB** parameter. You can select more than one channel.

   There are five channels of support. The default is Bit 0: INFO.

   - Bit 0:INFO (informational messages)
   - Bit 1:ERROR (error conditions)
   - Bit 2:WARNING (warning conditions)
   - Bit 3:NORMAL (debug level information)
   - Bit 4:VERBOSE (more detailed information than NORMAL)

For example:

- To enable only the INFO channel: 0x01.
- To enable only the ERROR channel: 0x02.
- To enable ERROR and WARNING channel: (0x02 | 0x04) = 0x06.
- To enable all channels: (0x01 | 0x02 | 0x04 | 0x08 | 0x10) = 0x1f.

7. Build your VIP.
8. Download the image and boot the target.

   Relevant track messages are output on the console.

3. Debugging with Message APIs

The error debug setting supports the error and failure messages that routines encounter. Information debug prints the information directly to the device.

Procedure

1. Add the debug macro for the EHCD module in **usbOsalDebug.h**.

   The format is **usbEhcd**, so the debug level should define **usbEhcdDebug** in **usrUsbDebug.c**. This should be an extern in **usbOsalDebug.h**, as below:

```
/* EHCD DEBUG */
#define USB_EHCD_ERR(fmt, arg1, arg2, arg3,
arg4, arg5, arg6)   \
    USB_ERR(usbEhcd, fmt, arg1, arg2, arg3,
arg4, arg5, arg6)
#define USB_EHCD_DBG(fmt, arg1, arg2, arg3,
arg4, arg5, arg6)   \
    USB_DBG(usbEhcd, fmt, arg1, arg2, arg3,
arg4, arg5, arg6)
#define USB_EHCD_VDBG(fmt, arg1, arg2, arg3,
arg4, arg5, arg6)  \
    USB_VDBG(usbEhcd, fmt, arg1, arg2, arg3,
arg4, arg5, arg6)
#define USB_EHCD_INFO(fmt, arg1, arg2, arg3,
arg4, arg5, arg6)  \
    USB_INFO(usbEhcd, fmt, arg1, arg2, arg3,
arg4, arg5, arg6)
#define USB_EHCD_WARN(fmt, arg1, arg2, arg3,
arg4, arg5, arg6)  \
    USB_WARN(usbEhcd, fmt, arg1, arg2, arg3,
arg4, arg5, arg6)
/* IMPORTS */

IMPORT UINT32  usbEhcdDebug;
```

2. Define **usbEhcdDebug** in **usrUsbDebug.c**.

   The default level for debug is 0, which turns off debug.

```
/* defines */

#ifndef USB_TRACK_EHCD
#define USB_TRACK_EHCD            0
#endif /* EHCD debug information */

/* globals */

UINT32  usbEhcdDebug = USB_TRACK_EHCD;
```

3. Add the debug macro to the source file.

   Make sure the header file **usbOsalDebug.h** is included in the source file. The **usrUsbDebug.c** is included by default when INCLUDE_USB is included in the VxWorks Image Project.

   Insert the different debug macros into the places where you need to show error, warning, normal, verbose, and information messages. For example, in EHCD source code we can add error check debug after memory allocate failed.

```
/* Initialize the global array */

    g_pEHCDData = (pUSB_EHCD_DATA *)OS_MALLOC
                  (sizeof(pUSB_EHCD_DATA) * USB_MAX_EHCI_COUNT);

/* Check if memory allocation is successful */

    if (g_pEHCDData == NULL)
        {
        USB_EHCD_ERR("usbEhcdInit - allocation failed for g_pEHCDData\n",
            0, 0, 0, 0, 0, 0);
        return;
        }
```

4. Define the USB_TRACK_EHCD debug level which you want printed in the project.

   The WARNING debug setting outputs the warning messages which routines meet, for example, the hub set configuration retried.

   - 0: OFF
   - 1: ERROR
   - 2: WARNING
   - 3: NORMAL
   - 4: VERBOSE

   The NORMAL debug setting prints the important values and checkpoints, for example, connect/disconnect interrupt complete, reset done, device and driver matched, and so on.

   VERBOSE debug outputs the common diagnostic messages such as submit an urb, callback coming and port status, and so on.

5. Rebuild the USB libraries with: **ADDED_CFLAGS+=-DUSB_DEBUG_ENABLE**
6. Build your VxWorks image project (VIP) and then run it.

3. MIGRATION AND PORTING TOPICS

1. Configuring Freescale Dual-Role USB Controllers

If you are using a legacy BSP with a pre-VxWorks 7 version of VxBus, you need to follow the configuration instructions in this topic.

About This Task

Freescale has integrated a Dual-Role USB controller IP into several of its SOC systems, with different chip specific configurations that enable various features of the IP. If you are using a legacy BSP with a pre-VxWorks 7 version of VxBus, follow the configuration instructions in this section. For the version of VxBus updated for VxWorks 7, you do not need to specifically configure BSP. You only need to add the desired device node in the ***.dts** file, following the instructions given in the appropriate BSP Supplement Guide for your BSP on the Wind River Support Network site.

Key features of the controller IP are described below.

- This IP can be configured in 3 modes: Host mode, Device mode, and OTG mode. There is a USBMODE register that can be configured by software to select the proper operating mode. On some SOCs, it can only be used as a host and is not configurable.
- In host mode, the IP works mostly as EHCI, but it has some extensions (variations) to EHCI so that it does not need a companion host controller (like UHCI or OHCI in traditional EHCI) to support FULL/LOW speed devices attached directly to the root ports. The extension is called "Embedded TT (Transaction Translator)" or "Integrated Root hub TT." Wind River USB EHCD has been extended to support this feature. See Configuring Embedded TT for EHCI Variations on page 9 for how to enable the "Embedded TT" feature for EHCI.
- In device mode, it operates as a fully functional USB 2.0 device controller. In previous VxWorks releases, Wind River has provided the target driver for this IP in each BSP that supports it, with BSP-dependent driver names. Wind River USB 2.5 integrates the driver into the USB stack as a library component, called INCLUDE_FSL_TCD.
- Different SOCs may have different register and descriptor endian formats, depending on the silicon configuration. This is not documented explicitly in Freescale manuals. If a new BSP is to be developed using this IP, BSP developers should check with Freescale support (or their reference implementations) to obtain the exact endian format of the IP implementation for specific SOCs. The Wind River USB stack has been implemented so that the endian modes for registers and descriptors are configurable by the BSP. All the BSPs delivered in VxWorks 6.8 and later that support this IP have been developed according to these configurations. See About Configuring Endian Formats for EHCI Variations on page 11 for how to set these configuration options in the host mode.

Note: This IP may have also been integrated by other silicon vendors into their chips. BSP developers should check if their target platform matches the features of this IP. If it matches, the same configuration procedures and parameters can be applied.

2. Configuring Embedded TT for EHCI Variations

Freescale Embedded Transaction Translator Function (also referred to as root hub TT) is used to support FULL/LOW speed devices directly attached to the EHCI root ports, but without handing control over to companion host controllers as with most standard EHCI controllers.

About This Task

EHCD has been improved to support root hub TT. Although there are many Freescale EHCI controllers with this feature, it is not common to all standard EHCI controllers. So, check your EHCI host controller manual to see if it has root hub TT support. This is

normally marked as “Embedded Transaction Translator Function” in a “Deviations from the EHCI Specifications” section of the manual.

BSP developers must provide configuration information, using the VXB_INST_PARAM_OVERRIDE parameter, to enable support for Embedded TT. If embedded TT is supported in the hardware, make an entry for hasEmbeddedTT in the sysInstParamTable parameter table, which is found in the file **hwconf.c**. See the example provided later in this section.

Also, note that some older model MPC834x EHCI controllers have an errata, #14, which specifies a workaround to fix the port number in the QH for the root hub TT support. There is an entry, **fixupPortNumber**, in the **sysInstParamTable** parameter table, for that purpose.

The errata is titled “USB14 Port numbering in Queue Head violates EHCI specification,” and is also marked as “Fixed in silicon revision 3.x.”

If you are working on very old hardware, remember to specify the **fixupPortNumber** entry as TRUE in the **sysInstParamTable** parameter table of **hwconf.c**.

You can use the routine below to check if the hardware has this issue:

```
BOOL usbEhcdFslPortNumberBug(void)
    {
    BOOL ret = FALSE;
    UINT32 chip, rev, svr;
    svr = vxSvrGet();
    chip = svr >> 16;

    rev = (svr >> 4) & 0xf;
    /* Erratum #14 on MPC834x Rev 1.0 & 1.1 chips */
    if ((rev == 1) && (chip >= 0x8050) && (chip <= 0x8055))
        {
        ret = TRUE;
        }

    return ret;
    }
```

3. Configuring OTG Dual Role Devices for EHCI Variations

Some USB hardware is capable of functioning as a USB OTG (On The Go) “dual role device,” which combines support for both host and device in a single module. However, Freescale hardware requires additional steps.

About This Task

Because the Freescale hardware has a non-EHCI compliant USBMODE register, it requires additional steps in order to take advantage of the OTG "dual role device" feature.

The Freescale USBMODE register defaults to “Idle” mode after a reset, that is neither host nor device mode. Wind River USB EHCD initialization code will reset the EHCI hardware module, thus the hardware will be in “Idle” mode for these “dual role devices.”

As a workaround to this, we have provided the entry postResetHook, which is intended to be provided by BSP developers to set the hardware into host mode. This is a function pointer; it will be directly called after the reset for the EHCI controller if it is provided as non-NULL.

Also, note that **postResetHook** is a generically named routine which developers can also use with other devices that might require this functionality. This function pointer is called after the USB OTG module reset.

BSP developers should check if your hardware is a “dual role device” which needs to be set to host mode after an EHCI reset.

Below is an example of the **sysInstParamTable** for the wrSbc834x **hwconf.c**. This target has embedded TT support, and is mounted with an MPC834x CPU, which has the port number errata in its EHCI hardware. Note that this target has only the host module, so we do not need to provide a hook to set to host mode after reset. With this information, write the **sysInstParamTable**, as shown below:

```
VXB_INST_PARAM_OVERRIDE sysInstParamTable[] =
{
{"vxbPlbUsbEhci",0,"hasEmbeddedTT", VXB_PARAM_INT32,{(void *)TRUE}},
{"vxbPlbUsbEhci",0,"fixupPortNumber", VXB_PARAM_INT32, {(void *) TRUE}},
{"vxbPlbUsbEhci",0,"postResetHook", VXB_PARAM_FUNCPTR, {(void *)NULL}},
{ NULL, 0, NULL, VXB_PARAM_END_OF_LIST, {(void *)0} }
};
```

4. About Configuring Endian Formats for EHCI Variations

If the descriptor or register endian format is different from the processor endian format, the host controller driver needs to swap the data to match the processor’s format when reading or writing to the descriptor or register.

Normal USB host controllers use little endian format for data structures and registers, but some processors (or some versions of the same processor) use big endian format for host controller data structures and registers (these are PLB based, not PCI based). One example is the Freescale MPC5121E USB host, which is a variation to the standard EHCI.

Since the exact descriptor or register endian format for such PLB based host controllers are chip specific, and cannot be presumed by the stack, Wind River USB provides the following parameters for configuring the endian formats from a BSP:

- the **descBigEndian** parameter passed by BSP
- the **regBigEndian** parameter passed by BSP

Refer to the processor documentation (or the reference BSP, if this information is not in the documentation) to determine if the processor uses big endian data.

Endian Format of EHCI Data Structures

Normal EHCI data structures are little endian, but on some processors, such as the MPC5121E, the data structures are big endian. If the CPU is big endian and it implements these data structures to be little endian, then you must swap the descriptors when reading from or writing to the descriptor elements. But if the CPU is big endian and it implements these data structures to also be big endian, then you do not need to swap (which is the case for all MPC5121E SOCs). In short, if the descriptor endian format is different from the CPU endian format, you must swap the descriptor.

A common abstraction is provided by the boolean parameter **descBigEndian**, which can be overridden by the BSP to get the endian format for these data structures. This boolean value defaults to FALSE. Currently, only the MPC5121E needs to override the default value to be TRUE in the **sysInstParamTable**. If during new BSP development you find a new EHCI variation that also uses big endian data structures, please refer to the fsl_ads51212 BSP for an example of how to set this boolean value.

Refer to the processor documentation (or the reference BSP, if this information is not in the documentation) to determine if the processor uses big endian data.

Endian Format of EHCI Registers

Normal EHCI is little endian in register format, but some versions of the MPC5121E and other processors use big endian register format. If the CPU is big endian and it implements EHCI registers in little endian, then you must swap the value when reading from or writing to the registers (as in the case of the MPC5121E REV1.5). But if he CPU is big endian, and it implements EHCI registers in big endian, then you do not need to do this swap.

If the register endian format is different with CPU endian format, you need to swap the register value.

A common abstraction is provided by the boolean parameter **regBigEndian**, which can be overridden by the BSP to get the endian format for registers. This boolean value defaults to FALSE, and currently some versions (newer than REV 1.5) of MPC5121E need to override the default value and set this parameter to TRUE in the **sysInstParamTable**. Some versions (previous to REV 1.5) need to use the value FALSE.

If during new BSP development you find an EHCI variation that also use big endian registers, refer to the **fsl_ads51212** BSP for an example of how to set this boolean value.

5. BSP Configuration Requirements for Use with GEN2 Target Stack

Additional configuration is required when configuring a BSP for use with the USB GEN2 target stack. You typically use the USB GEN2 target stack when implementing OTG capabilities.

To encapsulate chip specific features in the BSP level, each BSP should include two parts:

- Basic routines related to the architecture, such as register read/write routines, USB PHY initialize/un-initialize routines, and so on.
- Chip resources, such as register base address, how many host channels in the chip can be used for transfer data, and so on.

For example, the BSP configuration resources for the MUSB MHDRC target mode driver may contain the following:

- **regBase** is the base address used to access the registers.
- **irq** is the IRQ number of the USB target controller.
- **irqLevel** is the IRQ level.
- **numDmaChannels** is the DMA channel count number.
- **numEndpoints** is number of endpoints.
- **useExternalPower** specifies whether it is using the external power.
- **dmaEnabled** specifies whether the DMA is enabled.
- **platformType** is the platform type.

6. When to Transition an HCD to Use vxbDmaBufLib

Wind River USB provides support for 64-bit builds. Changes may be required to make to previously-built applications work properly in 64-bit VxWorks environments.

All existing host controller drivers, including xHCI, OHCD, UHCD, and EHCD have been converted to use **vxbDmaBufLib** to make them support both 32-bit and 64-bit VxWorks. See the VxWorks 7 BSP and Driver Guide for information about **vxbDmaBufLib**. If your host controller hardware has one of the following features or limitations, you may need to transition the HCD to use **vxbDmaBufLib**.

Bus Controller with Memory Window Limitation

Some parent bus controllers have a memory window limitation that makes the host controller unable to see all system memory space. For example, the ixdp425 PCI bus controller has address window limitations, where the devices on the PCI bus cannot see all of the CPU system memory address space. For such systems, if a piece of memory that is allocated from the user (or from the USB stack itself) is out of the BSP configured bus address window, then the buffer cannot be seen by the USB host controller which is sitting on the PCI bus, thus the transfer cannot be correctly performed. This kind of address window limitation should be solved by a mechanism called bounce buffering. The **vxbDmaBufLib** functions can be used to provide this kind of bounce buffering.

If your USB host controller is sitting behind such a parent bus controller, then you may need to consider transitioning the HCD to use vxbDmaBufLib (otherwise, you may have to work around the memory window limitation by restricting all user transfer buffers in the range that can be seen by your host controller, or do bounce buffering by yourself).

32-Bit Physical Memory Limits

If your host controller can address only 32 bit physical memory space but you want to use it on 64-bit VxWorks, then you must transition the HCD to use **vxbDmaBufLib**.

VxWorks 64 bit systems require bounce buffering to support 32 bit hardware in 64 bit environments.

As a special case for the "memory window limitation problem" as listed above, the 64-bit VxWorks brings in a generic problem for 32 bit-only USB host controllers: how could a normal 32 bit only USB host controller work under a 64 bit system? A normal 32 bit USB host controller, such as the old UHCI and OHCI, and some EHCI host controllers, can only address the first 4GB of physical memory space, but on a 64 bit system there can be far beyond the 4GB physical memory available. Plus, on VxWorks 64 bit, the virtual memory layout can cause applications to pass virtual memory that is not 1-1 mapping with physical memory, thus the address translation between virtual memory space, physical memory space and bus memory space will certainly occur. The **vxbDmaBufLib** provides bounce buffering support for such a case, and also does proper address transition operations.

Summary

If a host controller will only be used in 32-bit systems and will not be used in VxWorks 64 bit systems, then the HCD can remain unchanged (that is, there is no need to change it to be **vxbDmaBufLib**-based). The USB stack will work as before.

If a host controller needs to work in both 32-bit and 64-bit systems or the host controller parent bus has any memory window limitation, then it has to change the HCD to use **vxbDmaBufLib**, as well as the proper LP64 adaptation.

7. About Transitioning to vxbDmaBufLib

If you need to use **vxbDmaBufLib** in an HCD, you should understand what a DMA tag is and what a DMA map is in the **vxbDmaBufLib** context.

A DMA descriptor, which is also used in the **vxbDmaBufLib** context, is a small chunk of structured memory, which describes some data transfers between the host controller and the system memory. A DMA descriptor has specific meaning in each host controller context; for example, the EHCI has several types of DMA descriptors, such as the QTD, QH; while in OHCI, a DMA descriptor may be the ED, TD. Sometimes, the documentation also uses "Transfer Descriptor", which also refers to the "DMA descriptor" concept.

For more information about **vxbDmaBufLib**, see the VxWorks 7 BSP and Driver Guide.

HCD Pipe Transfer Management on page 14 All USB data transfers are endpoint based; an endpoint is abstracted as a pipe in software. The pipe structure is host controller (HC) specific and is created by the HCD through the createPipe( ) interface.
Support for New HCD Pipe Control interface on page 16 There are two situations that require the HCD to provide a pipe control interface.
USB HCD vxbDmaBufLib Transition Example on page 19EHCD is uses as an example of how to use the DMA tag and DMA map attributes.

EHCI Controllers That Support 64-Bit Addressing Mode on page 19EHCI controllers supporting 64-bit addressing mode can use either transfer data structures or normal user data buffers.
EHCI Controllers That Support Only 32-Bit Addressing Mode on page 21EHCI controllers that support only 32-bit addressing mode can use either a transfer data structure or a normal user data buffer.
Comparison of Support for 32-Bit and 64-Bit Addressing with EHCI on page 22 This example illustrates using EHCI as a HCD with **vxbDmaBufLib**.

7.1. HCD Pipe Transfer Management

All USB data transfers are endpoint based; an endpoint is abstracted as a pipe in software. The pipe structure is host controller (HC) specific and is created by the HCD through the createPipe( ) interface.

The pipe structure will normally contain information such as device address, endpoint address, endpoint type, endpoint direction, maximum packet size, and other information specific to the HC. The information contained in the pipe is used to organize transfer descriptors when any transfer is initiated.

The default control pipe of a device is the communication channel to communicate basic configuration information of the device, such as configuration, status, and control information. When a USB device is first attached to a USB port, it will only respond to device address 0 control requests. The host stack will use that device address 0 and endpoint 0 to retrieve basic device descriptors and set a BUS (each HC represents a BUS) unique device address to the device, so that later communications can be made through that unique device address. For that purpose, each USB host controller (HC) instance has a per HC global default control pipe for device address 0 and endpoint number 0, to represent the initial default control pipe communications.

When the device is set to a unique device address, the default control pipe of the device will be independently treated, so it will have its own pipe control information to work on.

Each normal operational endpoint also has a pipe structure to keep track of the transfer information. We will need to consider the default control pipe and other normal endpoints differently based on the transfer specifics of the different endpoint types.

Default Control Pipe Considerations

A control transfer is split into 3 phases: SETUP/DATA/STATUS. The transfer is based on the 8 byte USB SETUP packet, which is described in detail in section “9.3 USB Device Requests” of the USB 2.0 specification. The host controller first sends a SETUP packet to the device, in the wLength field (which is of type UINT16) of this packet; it specifies the length of data that will be transferred in the DATA phase of this control transfer (if 0 is specified, there is no DATA phase). After the DATA phase, there will be a STATUS phase which has no actual data transfer (a zero length transfer).

Based on these transfer characteristics, each control transfer requires the following resources:

- One DMA tag for the DMA descriptors for all the 3 phases. These DMA descriptors typically will have specific alignment requirements (for example, the EHCI qTD needs to be aligned on a 32-byte boundary).
- One DMA tag for the data buffers (SETUP buffer and DATA buffer). These buffers will typically have no alignment requirement.
- One DMA map for the SETUP buffer.
- One DMA map for the DATA buffer.
- One DMA descriptor for the SETUP packet (for example, the EHCI qTD).
- Variable number of DMA MAPs for the DMA descriptors of the DATA buffer.
- Variable number of DMA descriptors for the DATA phase. The number of DMA descriptors is calculated according to the **wLength** field of the SETUP packet and maximum packet size of the control endpoint.
- One DMA descriptor for the STATUS packet.

The question comes as to how we treat the DATA phase. The DATA phase may actually not exist at all, or may exist with a variable length as specified by the **wLength** field of the SETUP packet. To make our implementation easier, we make a trade off by using a static MAX possible transfer size that can be accommodated in an UINT16, that is 65536 bytes, to create the DMA map for the DATA

buffer. This value will be set to the **maxSize** of the DMA tag for the DATA buffer. This is for simplicity and performance, as in this way we can reduce the many instances of creating/deleting different sized DMA MAPs in various places.

The trade off is that we create a DMA map with the assumption for a MAX possible DATA transfer size for the DATA phase of the control transfer. With these considerations, the following will be done in the createPipe( ) interface of each HCD:

- Create one DMA tag for the control pipe DMA descriptors.
- Create one DMA tag for the control pipe SETUP and DATA buffers.
- Create one DMA map for the control pipe SETUP buffer.
- Create one DMA map for the control pipe DATA buffer.
- Create the DMA descriptors using the DMA tag created for the control pipe DMA descriptors by using vxbDmaBufMemAlloc( ). Note that vxbDmaBufMemAlloc( ) will automatically create the associated DMA MAPs for these DMA descriptors.

  Then in the submitURB( ) interface of each HCD, load the SETUP and DATA buffers by using the vxbDmaBufMapLoad( ) and then fill these loaded buffer fragments into the DMA descriptors. Also, calls to the proper vxbDmaBufSync( ) or vxbDmaBufMapSync( ) will be done.

  When the URB is completed (or error occurs or cancel happens), the proper vxbDmaBufSync( ) or vxbDmaBufMapSync( ) will be done and then vxbDmaBufMapUnload( ) will be called for the DMA MAPs.

  The default control pipe considerations can apply for both the per HC global default control pipe and the per device instance default control pipe.

Normal Data Pipe Considerations

Normal data endpoints (pipes), such as Bulk, Isochronous, and Interrupt transfer types, have different transfer characteristics than the control pipe. The transfer buffer for the endpoint of these transfer types is supplied by the user (client software such as a class driver), and then split into an array of smaller chunks according to the maximum packet size of the endpoint. Each chunk will need a DMA descriptor. For such a pipe, we need the following resources when a transfer is initiated:

- One DMA tag for the transfer buffer, which typically will have no alignment requirement. The **maxSize** of the DMA tag is further discussed below.
- One DMA map for the transfer buffer.
- One DMA tag for the DMA descriptors, which typically will have alignment requirement (check your host controller documentation).
- Variable number of DMA MAPs for the DMA descriptors of the transfer buffer.
- Variable number of DMA descriptors for the transfer buffer, the number of DMA descriptors is calculated according to the length of the transfer buffer and maximum packet size of the endpoint.

The original USB stack HCD creates the DMA descriptors when a transfer request is initiated. For example, for a transfer URB (USB Request Block), the usbEhcdSubmitURB( ) will call usbEhcdCreateQTDs( ) to create the required number of qTDs for the Bulk and Interrupt transfer buffer. The transfer buffer addresses are filled into the qTDs respectively. Once the transfer completes, the qTDs are then freed. This kind of design is straightforward and easy to implement, and performs on-demand transfer allocation, but it has a drawback in that the DMA descriptors are allocated and freed dynamically, which could have some performance penalty.

In fact, most USB classes use the endpoints in a serialized manner, so that it should wait for the previous transfer to complete before it initiates the next transfer. In this case, there will be only one transfer underway at a time for a specific endpoint. In some cases, there may be several transfers pending for one endpoint, for example, for the Isochronous endpoint, there may be several URBs submitted for one endpoint simultaneously.

With the above considerations, we will change the current implementation so that the DMA descriptors are allocated only once when they are first required, and when the request is complete the DMA descriptors are not destroyed but are kept for later use without reallocation.

The HCD itself understands the DMA limitations for the specific pipe (normally, a host controller hardware has the same user buffer DMA requirement with all the endpoint types, but there may be some others that have different DMA limitations on different transfer

types, thus we need to have one DMA tag for each pipe). Each pipe will have a different maximum request size put forward by the class drivers which will be used to create the DMA tag.

Although the HCD itself understands the DMA limitations for the specific pipe, it does not know how many data bytes will be requested to be transferred by the user (client software such as class drivers). The class drivers will be responsible for providing such information, which will be set to the **maxSize** of the DMA tag. The **maxSize** of the DMA tag will typically be used for creating the bounce buffer when a DMA map is created. DMA MAPs for the pipe are created at device attaching time when the class drivers initially request to transfer buffers (a new USBD API usbHstPipePrepare( ) is provided for class drivers to set up the possible data transfer requirement for each endpoint; see details in Support for New HCD Pipe Control interface on page 16. A DMA map corresponds to a URB transfer buffer. We have the following relations:

- A device may have several endpoints, including at least one control endpoint.
- An endpoint corresponds to a pipe data structure in HCD. A class driver may simultaneously submit several URBs (USB Request Blocks) to one pipe.
- Each URB will have a corresponding request info structure which contains HCD specific information. Thus each pipe also may have several request info structures.
- A pipe will have a DMA tag created, with the **maxSize** set to the value specified to the maximum possible transfer buffer size in all the URBs that can be submitted by a class driver to the pipe.
- Each request info structure will have a DMA map created, based on the DMA tag of the pipe; the DMA map contains the possible bounce buffer, with the size to be at least **maxSize**.
- Each request info structure (in fact the corresponding URB buffer) may need several “transfer descriptors” (DMA descriptors in **vxbDmaBufLib** context).

Parent topic: About Transitioning to vxbDmaBufLib on page 13

7.2. Support for New HCD Pipe Control interface

There are two situations that require the HCD to provide a pipe control interface.

Class Drivers Need an API to Setup Transfer Characteristics

USB class drivers will have two types of transfer requests mechanisms (the way URBs are submitted and results are waited):

- Single transfer request per time and wait for URB completion before submitting next URB. Typically used for serialized transport protocols such as the USB mass storage class, where previous transfer results have significant effects on later transfers.
- Continuous transfer requests without waiting for URB completion. Typically used for streaming transport protocol such as the USB audio Isochronous transfers where the previous transfer result does not affect the later transfers.

To support creating of DMA tag and DMA map for a pipe by class drivers for the above transfer requests mechanisms, the USBD API usbHstPipePrepare( ) is provided.

```
/*******************************************************************************
* usbHstPipePrepare - prepare a pipe for data transfers
*
* This routine is used to prepare a pipe for data transfers, by specifying the
* maximum possible transfer request size and the maximum number of simultaneous
* requests for one pipe. Based on the maximum possible transfer request size,
* the HCD can create the underlying DMA tag; based on the maximum number of
* possible transfer requests at a time, the HCD can create enough DMA MAPs
* to accommodate all simultaneous pending transfer requests.
*
* RETURNS: USBHST_SUCCESS, or other USBHST_STATUS values if fails.
*
* ERRNO: N/A
```


WNDRVR

```
*/

USBHST_STATUS usbHstPipePrepare
    (
    UINT32                      hDevice,      /* USB device handle */
    UINT8                       uEpAddr,      /* USB endpoint number */
    pUSB_TRANSFER_SETUP_INFO    pSetupInfo    /* Pipe transfer setup info */
    );

The USB_TRANSFER_SETUP_INFO structure is defined in target/h/usb/usbHst.h

typedef struct usb_transfer_setup_info
    {
    size_t   uMaxTransferSize; /* Maxium possible transfer size of this pipe */
    size_t   uMaxNumReqests;   /* Maxium possible transfer requests at a time */
    UINT32   uFlags;           /* Any possible flags */
}USB_TRANSFER_SETUP_INFO, *pUSB_TRANSFER_SETUP_INFO;
```

The **uMaxTransferSize** specifies the maximum transfer request size that can be made in a URB. This must be provided by the class drivers with the class driver specific knowledge of how large its requests would be in all their latter calls to the usbHstURBSubmit( ) API. The **uMaxTransferSize** will be used by the HCD to allocate proper bounce buffer (using the **vxbDmaBufLib**) in the preparation of the case where the user buffer need to be copied to/from the bounce buffer for real transfer to address the issues prescribed in the overview section of this document. Because of this, the HCD will not allow any latter calls to usbHstURBSubmit( ) API to pass a URB with its **uTransferLength** field bigger than the **uMaxTransferSize** value. Doing so would exceed the underlying bounce buffer thus is dangerous.

As described before, the default pipe has a special treatment in that we will always set this value as 65536, which is the biggest value that can be made in a UINT16 wLength field of the SETUP packet of the control transfer.

The **uMaxNumReqests** specifies the maximum number of simultaneous transfers that can be made for a pipe. Based on this value, the HCD will allocate this number of corresponding request info structures to be used for later requests submitted to this pipe. These request info structures will initially be placed on the free request list of the pipe; when new URB is submitted, the HCD will take one free request info from the free request info list and put it onto the active request list of this pipe; when the request is completed it will then be put back to the free request list of this pipe. If the class driver submits more simultaneous requests than are specified in the **uMaxNumReqests** value, the HCD has two options:

1. Reject this request because the class driver should know how many simultaneous requests it will submit.
2. Allocate new request info structure and then allow the requests to be made using this new request info structure, and then when this request is complete, we can add this request info structure into the list of free request info structures and allow it to be used later. In this regard, the **uMaxNumReqests** value is essentially the "default number of simultaneous requests for the pipe".

   The implementation of the EHCD is to use the option described in the next section.

USBD Needs a New Way to Implement Pipe Reset Functionality

The original implementation of usbdResetPipe( ) will firstly delete the pipe and then create the pipe again when usbHstClearFeature( ) is called. The pipe deletion will delete all the resources and cancel all the pending requests for the pipe. The pipe creation will then create again for all the resources. The exact effect of the delete and create is to make sure the host side endpoint associated with the pipe return to the DATA0 state when the STALL feature is cleared. In fact, we do not need to delete all the resources and create them again in such a case; we just need to cancel all the pending requests on that pipe and reset the data toggle to DATA0 state when resetting a pipe.

With the introduction of the DMA tag and DMA MAPs associated with the pipe, if we delete the pipe, then the DMA tag and DMA maps information is lost (because these information is only maintained by the HCD).

WNDRVR

Due to the above reason, we need another way to implement usbdResetPipe( ), which does only the necessary steps to cancel all the pending requests of a pipe and reset the data toggle pipe to DATA0; it does not destroy other resources of the pipe, especially the DMA tag and DMA maps, which will be used for latter transferring.

To support the above USBD APIs, a new HCD interface, pipeControl( ) is added into the **USBHST_HC_DRIVER** structure.

```
/* This structure contains the HC driver function pointers */
typedef struct usbhst_hc_driver
    {
    /* Number of bus for this host controller */
    UINT8       uNumberOfBus;
…
        /* Function pointer to control the pipe characteristics */
        USBHST_STATUS (*pipeControl) (UINT8 uBusIndex,
        ULONG uPipeHandle,
        pUSBHST_PIPE_CONTROL_INFO pCtrlInfo);
        …
        } USBHST_HC_DRIVER, *pUSBHST_HC_DRIVER;
```

The pipeControl( ) routine can be used to support both the usbHstPipePrepare( ) and the usbdResetPipe( ) functionality. Thus an HCD implementation for the pipeControl( ) can take the following structure:

```
/* Extract the USB_EHCD_PIPE data structure */

    pHCDPipe = (pUSB_EHCD_PIPE)uPipeHandle;

    switch (pPipeCtrl->uControlCode)
        {
        case USBHST_CMD_PIPE_RESET:
            {
            /*
             * Do pipe reset so that the pipe will be in DATA0 state
             * in next transfer
             */
            }
            break;
        case USBHST_CMD_TRANSFER_SETUP:
            {
            /*
             * Set up the transfer characteristics such as
             * creating proper DMA tag and DMA MAPs
             */
            }
            break;
        default : break;
        }
```

The **USBHST_PIPE_CONTROL_INFO** is defined as below (in the *installDir* **/target/h/usb/usbHst.h**):

```
/* This structure is used to store the pipe control parameters */

typedef struct usbhst_pipe_control_info
    {
    UINT8   uEndPointAddress; /* Endpoint address of the pipe */
    UINT32  uControlCode;     /* Control code that is to be operated */
    ULONG   uControlParam;    /* Control parameter */
```

```
    USBHST_STATUS   nStatus;  /* Control status */
}USBHST_PIPE_CONTROL_INFO, *pUSBHST_PIPE_CONTROL_INFO;
```

Currently, we have defined two control codes, **USBHST_CMD_PIPE_RESET** and **USBHST_CMD_TRANSFER_SETUP**, which support usbdResetPipe( ) and usbHstPipePrepare( ), respectively. If you are developing your own HCD, you could refer to the EHCD for an example implementation for this interface.

Parent topic: About Transitioning to vxbDmaBufLib on page 13

7.3. USB HCD vxbDmaBufLib Transition Example

EHCD is uses as an example of how to use the DMA tag and DMA map attributes.

The EHCD is selected as an example because the EHCI has a feature that it may support both 32 bit and 64 bit addressing modes, which is perfect for demonstrating how 32 bit only host controllers and 64 bit host controllers can be supported by **vxbDmaBufLib**.

Supporting Both 32 Bit and 64 Bit Addressing Modes of EHCI

Some EHCI host controllers only support addressing 32 bit memory space. However, the EHCI specification has optional support for true 64 bit addressing if it is running on a 64 bit system. To address the 64 bit memory space, we have to configure the EHCI properly. It works as follows:

- **HCCPARAMS** (Host Controller Capability Parameters) register has a register bit "64-bit Addressing Capability" to indicate it has the 64 bit addressing range capability of this implementation.
- **CTRLDSSEGMENT** (Control Data Structure Segment Register) register is a 32-bit register that corresponds to the most significant address bits [63:32] for all EHCI data structures. If the 64-bit Addressing Capability field in **HCCPARAMS** is a zero, then this register is not used. Software cannot write to it and a read from this register will return zeros. If the 64-bit Addressing Capability field in **HCCPARAMS** is a one, then this register is used with the link pointers to construct 64-bit addresses to EHCI control data structures. This register is concatenated with the link pointer from either the **PERIODICLISTBASE**, **ASYNCLISTADDR**, or any control data structure link field to construct a 64-bit address. This register allows the host software to locate all control data structures within the same 4 Gigabyte memory segment.
- Each of the transfer data structures has a corresponding 64 bit version, which has an extended area of 32 bit fields where the most significant address bits [63:32] for all the transfer buffers are filled. In this manner, the data transfer can access the whole 64 bit address space.

The "Extended Buffer Pointer Page X [63:32]" fields in the 64 bit version are filled by HCD with the most significant address bits [63:32] of transfer buffer pointers. The 64 bit addressing mode described above has two implications:

- All the transfer data structures must be located in the same 4GB memory space, based at the **CTRLDSSEGMENT** (Control Data Structure Segment Register) register.
- All the data transfer buffer can be true 64 bit.

Parent topic: About Transitioning to vxbDmaBufLib on page 13

7.4. EHCI Controllers That Support 64-Bit Addressing Mode

EHCI controllers supporting 64-bit addressing mode can use either transfer data structures or normal user data buffers.

If the EHCI host controller implementation supports 64 bit addressing mode, then the **addressMode64** field in the **USB_EHCD_DATA** structure instance will be set in usbHcdEhciDeviceConnect( ). When the **addressMode64** field is set, there are two options to choose from for the 64-bit addressing mode.

Transfer Data Structures (DMA descriptors) for 64-Bit Addressing Mode EHCI

The transfer data structures are in fact DMA descriptors, which are small chunks of memory that will be accessed by both HCD and the HC hardware. A good mechanism in **vxbDmaBufLib** for allocating this kind of memory is vxbDmaBufMemAlloc( ), which creates a DMA map based on the DMA tag created and then allocates memory with the attributes specified in the DMA tag. This section describes some important facts that affect creation of the DMA TAGs for the DMA descriptors, that is, the **lowAddr** and **highAddr** parameters to vxbDmaBufTagCreate( ).

In theory, if the EHCI supports 64 bit addressing mode, then the **CTRLDSSEGMENT** register can be programmed with any value, spanning the data structures across the whole 64 bit space, but within a contiguous 4GB segment. However, it is hard to implement and manage, because we are not sure where a normal memory allocation will be located in the 64 bit memory space. Vxworks 64-bit has a dedicated DMA32 memory partition, which is created primarily for the case where bounce buffer is needed for a user buffer beyond the 4GB limit but the hardware is not able to reach that limit. The DMA32 memory partition has a feature that all memory allocated from it is located physically in the first 4GB of the 64 bit memory space. This is the only way that we can get a bounded range of 4GB memory (located in the first 4GB of 64 bit memory space).

Based on the reasoning above, the **CTRLDSSEGMENT** register will be filled with value 0, meaning that all transfer data structures are in the lowest 4GB memory space. This choice is easier to implement and conforms to the VxWorks 64-bit DMA32 usage model.

The trade off is that even though the **CTRLDSSEGMENT** register can be programmed with any value, we limit it to 0 so that we can use the VxWorks 64-bit DMA32 memory for DMA descriptors.

- **CTRLDSSEGMENT** register will be filled as 0, which means all transfer data structures will be located in the first 4GB of the 64 bit memory space. We end up using the DMA32 memory for all transfer data structures, such as **qTD** structures.
- Use **lowAddr** as **VXB_SPACE_MAXADDR_32BIT**, and **highAddr** as **VXB_SPACE_MAXADDR** to call vxbDmaBufTagCreate( ) to create the DMA TAGs for these transfer data structures (DMA descriptors).

The next parameters to vxbDmaBufTagCreate( ) to consider are **alignment** and **boundary**. The EHCI transfer data structures (DMA descriptors), such as QH, qTD, iTD, siTD, FSTN, all have alignment requirements. Thus we should also set the **alignment** parameter to vxbDmaBufTagCreate( ) properly.

Also, the EHCI specification indicates that "software must ensure that no interface data structure reachable by the EHCI host controller spans a 4K page boundary", thus the **boundary** parameter will be set to 4096 when vxbDmaBufTagCreate( ) is called.

The questions comes to what value do we set to the **alignment** parameter so that the memory allocated can meet both boundary and alignment limitations. These alignment requirements for these data structures are no more than 64 bytes for 64 bit addressing mode EHCI controllers. But there is the fact that some of these data structures are bigger than 64 bytes, for example, the 64 bit version of the iTD is 92 bytes in size. If that data structure starts just 64 bytes off the end of the 4KB page boundary, then it may cross the 4KB boundary, which violates the EHCI specification.

To overcome the above problem, we can set the **alignment** parameter to be larger than the maximum 64 bit transfer data structures size(92), and we use 128 as the value for the **alignment** parameter.

The final parameters to vxbDmaBufTagCreate( ) to consider is **flags**. Since the transfer data structures are accessed by both HCD and HC, we can use cache safe buffer for them. In a 64 bit system, as described above, we will use **DMA32** memory partition for these transfer descriptors. The DMA32 partition itself is cache safe memory. So the **flags** parameter has no explicit effect on DMA descriptors on 64 bit systems.

However, to keep compatibility with 32 bit systems, where no DMA32 partition is available, and the memory allocated by vxbDmaBufMemAlloc( ) will be based on the **flags** parameter indication, we can set the **flags** parameter to **VXB_DMABUF_NOCACHE**, which will cause cacheDmaMalloc( ) to be used by vxbDmaBufMemAlloc( ) and the memory allocated will still be cache safe.

Also, we are sure that for these DMA descriptors there will be no bounce buffer that will need to be created because we have created them using DMA32 partition, and in every case the memory will be created cache safe. Thus, we do not need to call vxbDmaBufSync( ) during the use of these data structures.

Normal User Data Buffer (URB buffer) for 64-Bit Addressing Mode EHCI

If the EHCI supports true 64 bit addressing mode, it can access the whole 64 bit memory space for the data transfer. So, for the user data transfer, we set both the **lowAddr** and **highAddr** parameters to vxbDmaBufTagCreate( ) set to **VXB_SPACE_MAXADDR**. This sets no limitation for the user buffers.

Because the EHCI controllers will have no **alignment** and **boundary** limitation for the user buffer, the **alignment** parameter to vxbDmaBufTagCreate( ) will be set to 1, and **boundary** parameter will be set to 0.

The above factors and settings have the implication that there is no bounce buffering that needs to be enabled for 64 bit capable EHCI. Thus, the DMA TAGs will be created with the **flags** bit set with **VXB_DMABUF_MAPCREATE_NOBOUNCE_BUF**.

Parent topic: About Transitioning to vxbDmaBufLib on page 13

7.5. EHCI Controllers That Support Only 32-Bit Addressing Mode

EHCI controllers that support only 32-bit addressing mode can use either a transfer data structure or a normal user data buffer.

If the EHCI host controller implementation does not support 64 bit addressing mode, then the addressMode64 filed in the USB_EHCD_DATA structure instance will not be set in usbHcdEhciDeviceConnect( ). When the **addressMode64** field is not set, there are two options to choose from.

Transfer Data Structures (DMA descriptors) for 32-Bit Addressing Mode EHCI

If the EHCI controller does not support 64 bit addressing mode, in a 64 bit system, we will still use the DMA32 memory for all transfer data structures, such as the **qTD** structure. This is accomplished by using **lowAddr** as **VXB_SPACE_MAXADDR_32BIT**, **highAddr** as **XB_SPACE_MAXADDR** to call vxbDmaBufTagCreate( ) to create the DMA TAGs for these transfer data structures (DMA descriptors).

The **CTRLDSSEGMENT** register cannot be written and will be read as 0 in this case (meaning it will be naturally in the first 4GB memory space by default).

If the EHCI does not support 64 bit addressing, the maximum hardware alignment requirement is 32 bytes. And the maximum transfer data structure is the QH data structure, which is 48 bytes in size. Similar to the discussion for the 64 bit addressing mode EHCI for **alignment** and **boundary** parameters, we can set the **alignment** to 64, and **boundary** set to 4096.

Similar to the discussion for the 64 bit addressing mode EHCI for **flags** parameter, to keep compatibility with 32 bit system, we can set the **flags** parameter to **VXB_DMABUF_NOCACHE**, which will cause cacheDmaMalloc( ) to be used.

The describer buffers will be allocated directly by calling vxbDmaBufMemAlloc( ). With the above factors and settings, there will be no bounce buffer allocated for these DMA descriptors.

Normal User Data Buffer (URB buffer) for 32-Bit Addressing Mode EHCI

If the EHCI does not support 64 bit addressing mode, it can only access the 32 bit memory space (or even smaller) for the data transfer. So, for the user data transfer, we use the following approach.

We will set **lowAddr** as **VXB_SPACE_MAXADDR_32BIT**, and **highAddr** as **XB_SPACE_MAXADDR**. On 64 bit systems, this will cause the bounce buffer to be created when vxbDmaBufMapCreate( ) is called. On 32 bit systems, there will be no bounce buffer created as in normal cases.

Because the EHCI controllers will have no **alignment** and **boundary** limitation for the user buffer, the **alignment** parameter to vxbDmaBufTagCreate( ) will be set to 1, and the **boundary** parameter will be set to 0.

The **flags** parameter will be set to 0.

Parent topic: About Transitioning to vxbDmaBufLib on page 13

7.6. Comparison of Support for 32-Bit and 64-Bit Addressing with EHCI

This example illustrates using EHCI as a HCD with **vxbDmaBufLib**.

For 64 bit addressing EHCI:

- For the data structures, we use 128 as the value for the **alignment** parameter.
- For the user data buffer, we use both **lowAddr** and **highAddr** set to **VXB_SPACE_MAXADDR**. The **flags** parameter will be set to **NOBOUNCE_BUF** on 64 bit systems.

For 32 bit addressing EHCI:

- For the data structures, use 64 as the value for the **alignment** parameter.

  For the user data buffer, set **lowAddr** as **VXB_SPACE_MAXADDR_32BIT**, **highAddr** as **XB_SPACE_MAXADDR**. The **flags** parameter will be set to 0.

Parent topic: About Transitioning to vxbDmaBufLib on page 13

8. Adapting APIs for Existing Class Drivers for New HCDs

Class drivers required changes in order to work with the HCDs that have undergone **vxbDmaBufLib** transition. All Wind River provided class drivers have already undergone these changes.

About This Task

If you have customized vendor provided class drivers, and want these class drivers to work with these HCDs which have undergone **vxbDmaBufLib** transition, then you have to apply the changes to your class drivers as described below.

If you do not want to change them, then these class drivers can only work on HCDs that are not converted to **vxbDmaBufLib** (that also means you have decided not to run on VxWorks 64 bit systems).

Once you have done the changes as described below, then the class drivers will run on both HCDs that have transitioned to **vxbDmaBufLib** and HCDs that have not transitioned to **vxbDmaBufLib**. The APIs will detect if the underlying HCD has the pipeControl( ) interface provided and if not, just ignore it and fall back to the **vxbDmaBufLib** non-transitioned HCD model.

Basically, the changes needed for a class driver are to enforce the class driver to limit its maximum transfer request size in each URB. You should decide the maximum transfer request size in each URB with respect to your class driver knowledge. For example, a printer class driver could limit its maximum transfer request size in each URB to be 8192 bytes. If a user requests more than that size, the user request is broken into multiple transfers of that size plus any remaining bytes and then URBs are submitted sequentially to transfer the entire user requested size. How big the “maximum transfer request size in each URB” is depends on your decision, but you should have one maximum.

You could also specify how many simultaneous URBs can be pending transfer on one pipe. Normally, it will be only one, but some class drivers may request multiple simultaneous URBs for one pipe. It still depends on class driver knowledge.

Calling usbHstPipePrepare( ) Before Initiating Transfers

GEN2 class drivers must call usbHstPipePrepare( ) before initiating transfers.

```
/*******************************************************************************
* usbHstPipePrepare - prepare a pipe for data transfers
*
* This routine is used to prepare a pipe for data transfers, by specifying the
* maximum possible transfer request size and the maximum number of simultaneous
* requests for one pipe. Based on the maximum possible transfer request size,
* the HCD can create the underlying DMA TAG; based on the maximum number of
* possible transfer requests at a time, the HCD can create enough DMA MAPs
* to accommodate all simultaneous pending transfer requests.
*
* RETURNS: USBHST_SUCCESS, or other USBHST_STATUS values if fails.
*
* ERRNO: N/A
*/

USBHST_STATUS usbHstPipePrepare
    (
    UINT32                      hDevice,      /* USB device handle */
    UINT8                       uEpAddr,      /* USB endpoint number */
    pUSB_TRANSFER_SETUP_INFO    pSetupInfo    /* Pipe transfer setup info */
    );
```

Providing cpuToBus/busToCpu Conversions

In the previous USB stack, the BSP had to provide cpuToBus/busToCpu address conversion stubs which convert the physical address between bus addresses. With the introduction of **vxbDmaBufLib**, all the virtual to physical and physical to virtual bus address conversions are done automatically by the library routines. Therefore, these resource entries are now not needed. However, existing BSPs can keep these entries, as there are no side effects. We do not query these entries.